

B28: SQL SuperHero Program : Session 29

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

6 April 2025 at 09:16

Function with 2 parameters:

```
create or replace function get_emp_salary (p_dept_id number, p_name varchar2) return number as
x_sal hr.employees.salary%Type;
Begin
    select salary into x_sal from hr.employees where department_id = p_dept_id and first_name = p_name;
    Return x_sal;
End;
```

Function created.



```
select get_emp_salary(90,'Neena') from dual
```

GET_EMP_SALARY(90,'NEENA')
17000

2 where clause = 2 input parameter

①

②

```
create or replace function get_emp_salary (p_dept_id number, p_name varchar2) return date as
x_sal hr.employees.hire_date%type;
Begin
    select hire_date into x_sal from hr.employees where department_id = p_dept_id and first_name = p_name;
    Return x_sal;
End;
```

Function created.

```
select get_emp_salary(90,'Neena') from dual
```

GET_EMP_SALARY(90, 'NEENA')
21-SEP-05

→ You can not
give size
error

```
create or replace function get_email(p_employee_id number) return varchar2(20) as
l_mail hr.employees.email%type;
Begin
    select email into l_mail from hr.employees where employee_id = p_employee_id;
    return l_mail;
End;
```

Errors: FUNCTION GET_EMAIL

Line/Col: 1/57 PLS-00103: Encountered the symbol "(" when expecting one of the following:

. @ % ; is default authid as cluster order using external
character deterministic parallel_enable pipelined aggregate
result_cache accessible rewrite

Line/Col: 2/12 PLS-00103: Encountered the symbol "HR" when expecting one of the following:

See the above code, when we write function definition line, we should not specify the size of any datatype. It gives errors. Either input parameter or output, size no need to be specified.

Any requirement, we can convert in function. like the below, I will pass employee_id and emp_name and I need department_name.
First write the sql and then convert in pl/sql. and then function

```
create or replace function get_dept_name(p_id number, p_name varchar2) return varchar2 as
```

```
l_name Hr.departments.department_name%type;
```

```
Begin
```

```
select department_name into l_name
from hr.employees E, Hr.departments d
where e.department_id = d.department_id
and e.employee_id = p_id
and e.first_name = p_name;
```

```
Return l_name;
```

```
End;
```

```
create or replace function get_dept_name(p_id number, p_name varchar2) return varchar2 as
```

```
l_name Hr.departments.department_name%type;
```

```
Begin
```

```
select department_name into l_name
from hr.employees E, Hr.departments d
where e.department_id = d.department_id
and e.employee_id = p_id
and e.first_name = p_name;
```

```
Return l_name;
```

```
End;
```

```
Function created.
```

→ Different data types

→ Basically your where clause

```
create or replace function get_dept_name(p_id hr.employees.employee_id%type, p_name varchar2) return varchar2 as
```

```
l_name Hr.departments.department_name%type;
```

```
Begin
```

```
select department_name into l_name
from hr.employees E, Hr.departments d
where e.department_id = d.department_id
and e.employee_id = p_id
and e.first_name = p_name;
```

```
Return l_name;
```

```
End;
```

```
Function created.
```

↓
Datatype of input → no need to say number/varchar/date
↓
You can define like this as well

①

②

```
create or replace function sudhakar(p_fname varchar2, p_lname hr.employees.last_name%type) return date is
```

```
l_date hr.employees.HIRE_DATE%Type;
```

```
Begin
```

```
select HIRE_DATE into l_date from hr.employees where first_name = p_fname and last_name = p_lname;
return l_date;
```

```
End;
```

```
Function created.
```

→ fn created with 2 input parameter

```
select sudhakar('Lex') from dual
```

→ passing only 1, when we call the function

ORA-06553: PLS-306: wrong number or types of arguments in call to 'SUDHAKAR'

← gives error

More Details: <https://docs.oracle.com/error-help/db/ora-06553>

Number of argument and datatype of arguments must be same.

```

create or replace function get_dept_name(p_id hr.employees.employee_id%type, p_name varchar2) return hr.employees.hire_date%type
l_name hr.departments.hire_date%type;
Begin
    select e.hire_date into l_name
    from hr.departments d, hr.employees e
    where d.department_id = e.department_id
    and e.employee_id = p_id
    and e.first_name = p_name;
return l_name;
End;

```

Errors: FUNCTION GET_DEPT_NAME
 Line/Col: 0/0 PL/SQL: Compilation unit analysis terminated
 Line/Col: 2/27 PLS-00302: component 'HIRE_DATE' must be declared

More Details: <https://docs.oracle.com/error-help/db/ora-24344>

} why this error came?

Some issue with declare

If you see above snapshot, see the error : "component 'HIRE_DATE' must be declared".

Let see the code : what I did : **l_name hr.departments.hire_date%type;**

hr.departments table does not have hire_date column.

=====

Function always return only 1 value.

```

create or replace function get_email(p_sal number) return varchar2 as
l_email hr.employees.email%type;
Begin
    select email into l_email from hr.employees where salary = p_sal;
return l_email;
End;

```

Function created.

```
select get_email(24000) from dual
```

GET_EMAIL(24000)
SKING

→ we have only 1 row with sal 24000 ✓

fn return only one value

`select get_email(17000) from dual`

2 record exist with 17000 salary

ORA-01422: exact fetch returns more than requested number of rows

More Details: <https://docs.oracle.com/error-help/db/ora-01422>

got the error

=====

=====

Write a function to return YES or NO to check data if exists.

Create or replace function is_region_exists (p_id number, p_name varchar2) return varchar as
l_count number;

l_exists Varchar2(10);

Begin

select count(*) into l_count from hr.regions where region_id = p_id and region_name = p_name;

if l_count > 0 then

l_exists := 'YES';

else

l_exists := 'NO';

End If;

Return l_exists;

End;

① ②

```

Create or replace function is_region_exists (p_id number, p_name varchar2) return varchar as
l_count number;
l_exists Varchar2(10);
Begin
    select count(*) into l_count from hr.regions where region_id = p_id and region_name = p_name;
    if l_count > 0 then
        l_exists := 'YES';
    else
        l_exists := 'NO';
    End If;

    Return l_exists;
End;

```

Function created.

The screenshot shows the following SQL code with handwritten annotations:

```

1 create or replace function get_region_name(P_region_id IN NUMBER, P_Region_name in Varchar2) return Varchar2 AS
2 l_exists Varchar2(20);
3 Begin
4 Select count(*) Into l_exists from hr.regions where region_id = P_region_id and REGION_NAME = P_Region_name;
5 If l_EXISTS > 0 Then
6 l_exists := 'YES';
7 Else
8 l_exists := 'NO';
9 End If;
10
11 Return l_exists;
12
13 End get_region_name;

```

Handwritten annotations in red and blue:

- Syntax** (red arrow pointing to the function definition line 1)
- Name (can be anything)** (red arrow pointing to the function name `get_region_name`)
- input Parameter (Basically where clause)** (black arrow pointing to the input parameters `P_region_id` and `P_Region_name`)
- what is output** (blue arrow pointing to the `Return l_exists;` line 11)
- Must needed else give error** (blue arrow pointing to the `Else` block lines 7-8)
- Satisfy the output** (red text at the bottom)

Function created.

Can I write 2 returns in a function?

```

create or replace function get_region_name(P_region_id IN NUMBER, P_Region_name in Varchar2) Return Varchar2 AS
L_exists Varchar2(20);
Begin

```

```

    Select count(*) Into L_exists from regions where region_id = P_region_id and REGION_NAME = P_Region_name;

```

```
If L_EXISTS > 0 Then
L_exists := 'YES';
Else
L_exists := 'NO';
End If;
Return P_region_id;
Return L_exists;
```

```
End get_region_name;
```

```
select get_region_name(1,'Europe') from dual;
```

--> **we can write 2 Return and function will compile. But it return the first RETURN. Return P_region_id;**

Here it return 1

and if we change the position of return ;like below:

```
create or replace function get_region_name(P_region_id IN NUMBER, P_Region_name in Varchar2) Return Varchar2 AS
L_exists Varchar2(20);
```

```
Begin
```

```
  Select count(*) Into L_exists from regions where region_id = P_region_id and REGION_NAME = P_Region_name;
```

```
  If L_EXISTS > 0 Then
  L_exists := 'YES';
  Else
  L_exists := 'NO';
  End If;
```

```
  Return L_exists;
  Return P_region_id;
```

```
End get_region_name;
```

--> here it will return the L_Exists . Return L_exists;

```
create or replace function get_sal(p_id number) Return number AS
L_salary Number;
Begin
  select salary into L_salary from hr.employees where employee_id = p_id;
  Return L_salary;
  Return p_id;
End get_sal;
```

→ 2 times RETURN

Function created.

→ NO PROBLEM

```
select get_sal(100) from dual
```

GET_SAL(100)

24000

→ Return the first RETURN

Let's Change the position of Return:

```
create or replace function get_sal(p_id number) Return number AS
L_salary Number;
Begin
  select salary into L_salary from hr.employees where employee_id = p_id;
  Return p_id;
  Return L_salary;

End get_sal;
```

Function created.

Change the position of
Return.

```
select get_sal(100) from dual
```

GET_SAL(100)

100

```

create or replace function get_sal(p_emp_id number) return number is
    l_sal number;
Begin
select salary into l_sal from hr.employees where employee_id = p_emp_id;
return p_emp_id;
select salary into l_sal from hr.employees where employee_id = 111;
return l_sal;
return 1;
End;

Function created.

select get_sal(100) from dual

```

→ After this nothing will run

GET_SAL(100)
100

Can we write dml in function?

[dml --> insert, update, delete]

YES. we can write a dml in function. it will compile and do exactly what is written in the function body. But it is not advisable because for computation oracle provides us Procedure.

Note: we can not call such a function in a select statement.

Create or replace function get_sal (p_emp_id in number) return number as

Begin

----- Our Executable statements

End;

This function takes emp_id as an input parameter and gives the salary of that employee.

we can use this function directly in a select. Like

```
select e.id, e.name, e.address, get_sal(e.id) salary from emp;
```

But suppose if we have a function like update_salary

create or replace function upd_sal (p_emp_id in number) return number as

Begin

< > -- executable statement contains DML

End;

This function takes emp_id as an input parameter and updates the salary of the employee.

```
create or replace function upd_sal(p_id number) return number as
```

```
  l_salary number;
```

```
Begin
```

```
  Update employees set salary = salary + 5000 where employee_id = P_id; → DML
```

```
  select salary into l_salary from employees where employee_id = P_id;
```

```
  return l_salary;
```

```
End;
```

```
Function created.
```

✓ its created.
(yes, we can write DML)

-> No issue in the writing function and it works exactly what is written.

we can not use that function in a select.

```
select e.id, e.name, e.address, upd_sal(e.id) salary from emp;
```

not allowed. it gives error



Calling f^u in a select

```
select upd_sal(100) from dual
```

ORA-14551: cannot perform a DML operation inside a query

More Details: <https://docs.oracle.com/error-help/db/ora-14551>

--> The way of calling such functions as follows:

x is number (because fn return number)

```
declare
x number;
Begin
  x := upd_sal(100);
End;
```

function called here

Return value in x

Statement processed.

```
select * from employees where employee_id = 100
```

updated from function

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY
100	Steven	King	SKING	515.123.4567	17-JUN-03	AD_PRES	24500

The same function is called here without any issue and it updates the salary of employee. so it means, we can write a dml operation inside function and it behaves exactly same, but only difference is , we can not use that function in a select statement.

Return the value here

```
declare
x employees.salary%type;
Begin
  x:= upd_sal(100);
  dbms_output.put_line(x);
End;
```

function called

Statement processed.
24500

function contains a DML statement

```
create or replace function upd_sal(p_employee_id number) return number as
l_salary employees.salary%type;
Begin
    update employees set salary = salary + 500 where employee_id = p_employee_id;
    -- select salary into l_salary from employees where employee_id = p_employee_id;
    return 1;
End;
```

→ must have to RETURN something.
Just let Return 1.

Function created.

return here

```
declare
l_salary employees.salary%type;
Begin
    l_salary:= upd_sal(100);
    dbms_output.put_line(l_salary);
End;
```

→ function called

Statement processed.

1

→ it was written to return 1.

```
select * from employees
```

Salary updated!

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY
100	Steven	King	SKING	515.123.4567	17-JUN-03	AD_PRES	25500